Cairo University Faculty
of Engineering
Computer Engineering Department

OS Scheduler

Document Structure

- Objectives
- Introduction
- System Description
- Guidelines
- Grading Criteria
- Deliverables

Objectives

- Evaluating different scheduling algorithms.
- Practice the use of IPC techniques.
- Best usage of algorithms and data structures.

Platform *Linux*

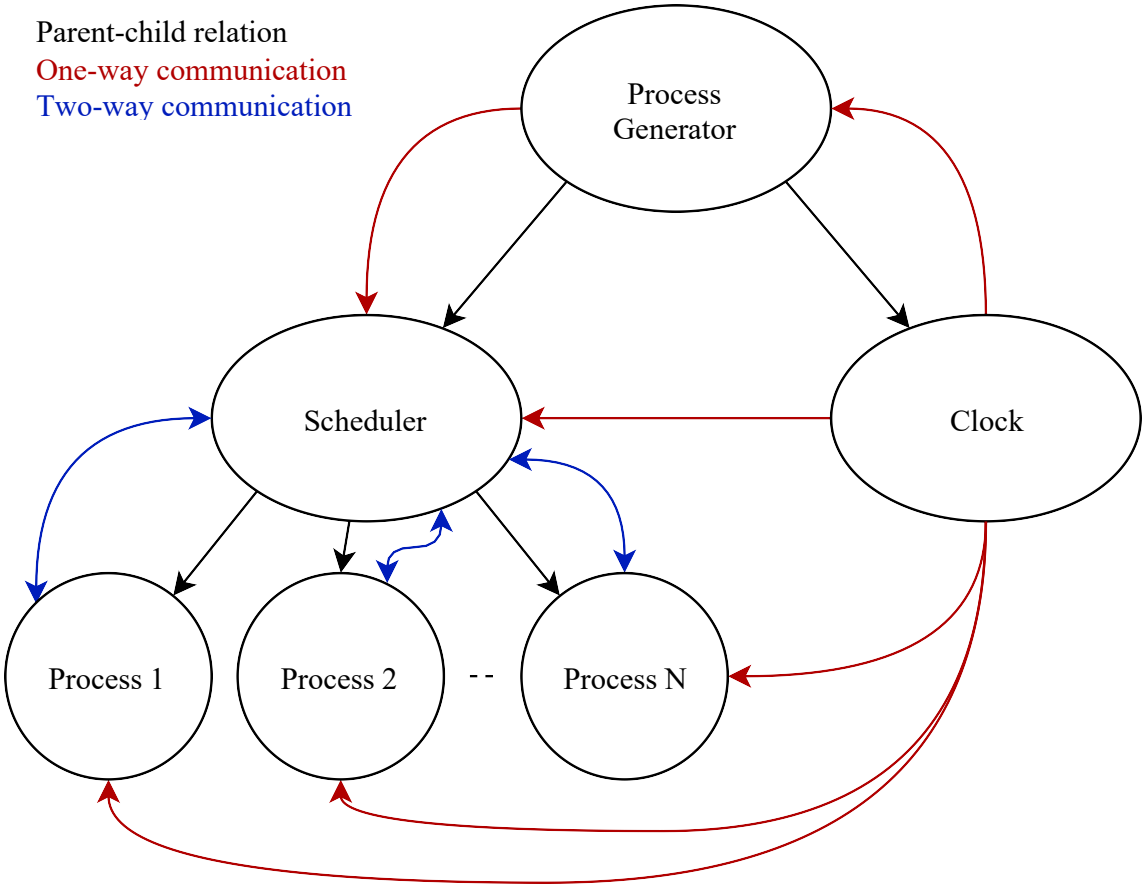
Language *C/C++*

Introduction

A CPU scheduler determines an order for the execution of its scheduled processes; it decides which process will run according to a certain *data structure* that keeps track of the processes in the system and their status.
A process, upon creation, has one of the three states: *Running*, *Ready*, *Blocked* (doing I/O, using other resources than CPU or waiting on unavailable resource).
A bad scheduler will make a very bad operating system, so your scheduler should be as optimized as possible in terms of memory and time usage.

System Description

Consider a computer with 1-CPU and infinite memory. It is required to make a scheduler with its complementary components as sketched in the following diagrams.



Part I: Process Generator (Simulation & IPC)

CODE FILE *process generator.c*

The process generator should do the following tasks...

- Read the input files (check the input/output section below).
- Ask the user for the chosen scheduling algorithm and its parameters, if there are any.
- Initiate and create the scheduler and clock processes.
- Create a data structure for processes and provide it with its parameters.
- Send the information to the scheduler *at the appropriate time* (when a process arrives), so that it will be put in its turn.
- At the end, clear IPC resources.

Part II: Clock (Simulation & IPC)

CODE FILE *clk.c*

The clock module is used to emulate an integer time clock. *This module is already built for you.*

Part III: Scheduler (OS Design & IPC)

CODE FILE *scheduler.c*

The scheduler is the core of your work; it should keep track of the processes and their states, and it decides - based on the used algorithm - which process will run and for how long.
You are required to implement the following THREE algorithms...

1. Preemptive Highest Priority First (HPF)
2. Round Robin (RR)
3. Expand your system to have a 2-CPU architecture, each CPU will have its own First Come First Serve (FCFS) scheduler with a stealing mechanism to balance the processes in the 2 CPUs

Part III (A): Process Assignment/Reassignment:

For the 2-CPU Architecture:

- Each CPU has a queue that holds processes.
- When a new process arrives, it is required to be added to one of the two queues (given to one of the two schedulers). Select the queue with **the least number of processes**. If both queues have the same number of processes, then add the new process to the queue of CPU #1 (give it to scheduler #1).
- After **N** clock cycles, you are required to compute the **total remaining time** of the processes in each queue. Then, calculate the difference between each number. If the **absolute difference** exceeds a threshold **M**, then you must **steal** a process from the **end** of the queue with the **higher total remaining time** and give it to the other queue, then perform the difference check again. It should keep doing this as long as the **absolute difference** exceeds **M**.

  NOTE 1: You **DO NOT** need to handle infinite loops (where the absolute difference never exceeds M. There will be no test cases that introduce this problem)

  NOTE 2: Process stealing has a constant overhead of **3 sec** for **BOTH CPUs** (Check the **Guidelines** section to know how you should treat the measuring unit of time).
- The previous step is done every **N** clock cycles
- N & M are parameters that you should ask the user for at the beginning.

Part III (B): Scheduling:

Any scheduling algorithm **(HPF, RR, FCFS)** schedules the processes in the *ready queue (Processes that have already arrived)* on one CPU only.

The scheduler should be able to do the following:

1. Start a new process. (Fork it and give it its parameters.)
2. Switch between two processes according to the scheduling algorithm. (Stop the old process, save its state, and start/resume another one.)
    - Switching has a constant overhead of 1 sec.
3. Keep a *process control block (PCB)* for each process in the system. A PCB should keep track of the state of a process: running/waiting, execution time, remaining time, waiting time, etc.
4. Delete the data of a process when it gets notified that it has finished. *When a process finishes, it should notify the scheduler on termination; the scheduler does NOT terminate the process.*
5. Report the following information
    - CPU utilization.
    - Average weighted turnaround time.
    - Average waiting time.
    - Standard deviation for the average weighted turnaround time.
6. Generate two files: (check the input/output section below)
    - Scheduler.log
    - Scheduler.perf

Part IV: Process (Simulation & IPC)

CODE FILE *process.c*

Each process should act as if it is CPU-bound.

Again, *when a process finishes, it should notify the scheduler on termination; the scheduler does NOT terminate the process.*

Part V: Input/Output (Simulation & OS Design Evaluation)
Part V (A): Input/Output for 1 CPU

Input File

processes.txt example

```
#id   arrival   runtime   priority
1     1         6         5
2     3         3         3
```

- Comments are added as lines beginning with # and should be ignored.
- Each non-comment line represents a process.
- Fields are separated with *one tab character* '\t'.
- You can assume that processes are sorted by their arrival time. *Take care that 2 or more processes may arrive at the same time.*
- You can use the *test generator.c* to generate a random test case.

Output Files

scheduler.log example

```
#At  time  x  process  y  state  arr  w  total  z  remain  y  wait  k
At  time  1  process  1  started  arr  1  total  6  remain  6  wait  0
At  time  3  process  1  stopped  arr  1  total  6  remain  4  wait  0
At  time  4  process  2  started  arr  3  total  3  remain  3  wait  1
At  time  7  process  2  finished  arr  3  total  3  remain  0  wait  1  TA  4  WTA  1.33
At  time  8  process  1  resumed  arr  1  total  6  remain  4  wait  5
At  time  12  process  1  finished  arr  1  total  6  remain  0  wait  5  TA  11  WTA  1.87
```

- Comments are added as lines beginning with # and should be ignored.
- Approximate numbers to the nearest 2 decimal places, e.g., 1.666667 becomes 1.67 and 1.33333334 become 1.33.
- Allowed states: *started*, *resumed*, *stopped*, *finished*.
- TA & WTA are written only at the *finished* state.
- You need to stick to the given format because files are compared automatically.

scheduler.perf example

```
CPU utilization = 75%
Avg WTA = 1.58
Avg Waiting = 3
Std WTA = 0.25
```

- If your algorithm does a lot of processing, processes might not start and stop at the same time instance. Then, your utilization should be less than 100%.
- For CPU utilization:
    - total actual CPU execution time = 6 + 3 = 9
    - total CPU running time = 12
    - CPU utilization = 9 / 12 = 75%

Part V (B): Input/Output for 2 CPUs

Input File

processes.txt example

#id	arrival	runtime	priority
1	0	10	1
2	0	9	2
3	1	8	3
4	2	6	4
5	3	1	5

- Comments are added as lines beginning with *#* and should be ignored.
- Each non-comment line represents a process.
- Fields are separated with *one tab character '\t'*.
- You can assume that processes are sorted by their arrival time. *Take care that 2 or more processes may arrive at the same time.*

Output Files (N=M=3)

scheduler_1.log example

```
#At  time  x  process  y  state  arr  w  total  z  remain  y  wait  k
At  time  0  process  1  started  arr  0  total  10  remain  10  wait  0
At  time  3  process  5  was  stolen
At  time  13  process  1  finished  arr  0  total  10  remain  0  wait  3  TA  13  WTA  1.30
At  time  14  process  3  started  arr  1  total  8  remain  8  wait  13
At  time  22  process  3  finished  arr  1  total  8  remain  0  wait  13  TA  21  WTA  2.63
```

scheduler_2.log example

```
#At  time  x  process  y  state  arr  w  total  z  remain  y  wait  k
At  time  0  process  2  started  arr  0  total  9  remain  9  wait  0
At  time  12  process  2  finished  arr  0  total  9  remain  0  wait  3  TA  12  WTA  1.33
At  time  13  process  4  started  arr  2  total  6  remain  6  wait  11
At  time  19  process  4  finished  arr  2  total  6  remain  0  wait  11  TA  17  WTA  2.83
At  time  20  process  5  started  arr  3  total  1  remain  1  wait  17
At  time  21  process  5  finished  arr  3  total  1  remain  0  wait  17  TA  18  WTA  18.00
```

scheduler_1.perf example

```
CPU utilization = 81.82%
Avg WTA = 1.96
Avg Waiting = 8.00
Std WTA = 0.66
```

Scheduler_2.perf example

```
CPU utilization = 76.19%
Avg WTA = 7.39
Avg Waiting = 10.33
Std WTA = 7.53
```

- **Explanation of .perf files**
  - **Scheduler 1**
    - Processes: P1, P3
    - Total runtime = 10 + 8 = 18
    - Last finish time = 22
    - Utilization = 18 / 22 = 81.82%
    - WTAs = 13/10 = 1.30, 21/8 = 2.625
    - Avg WTA = (1.30 + 2.625) / 2 = 1.9625 -> 1.96
    - Avg Waiting = (3 + 13) / 2 = 8.00
    - Std WTA = 0.6625 -> 0.66
  - **Scheduler 2**
    - Processes: P2, P4, P5
    - Total runtime = 9 + 6 + 1 = 16
    - Last finish time = 21
    - Utilization = 16 / 21 = 76.19%
    - WTAs = 12/9 = 1.3333, 17/6 = 2.8333, 18/1 = 18
    - Avg WTA = 7.3889 -> 7.39
    - Avg Waiting = (3 + 11 + 17) / 3 = 10.33
    - Std WTA = 7.5281 -> 7.53

Guidelines

- Read the document carefully at least once.
- You can specify any other additional input to algorithms or any assumption, but after taking permission from your TA.
- The user should be able to choose between different scheduling algorithms.
- You should specify how your algorithm handles ties.
- Priority values range from 0 to 10, where *0 is the highest priority and 10 is the lowest priority*.
- Your program must not crash.
- You need to release all the IPC resources upon exit.
- The measuring unit of time is 1 sec; there are no fractions, so no process will run for 1.5 seconds or 2.3 seconds. Only integer values are allowed.
- You can use any IDE (Eclipse, Code::Blocks, NetBeans, KDevelop, CodeLite, etc.) you want, of course, though it would be a good experience to use make files, standalone compilers, and debuggers if you have time for that.
- Spend a good time in design, and it will make your life much easier in implementation.
- The code should be clearly commented, and the variable names should be indicative.

Grading Criteria

- Non-compiling code = ZERO grade.
- Correctness & understanding (50%).
- Modularity, naming convention, and code style (20%).
- Design complexity & data structures used (20%).
- Teamwork (10%).

Deliverables

You should deliver code files, test cases, and a report containing the following information:

- Data Structure used.
- Your algorithm explanation and results.
- Your assumptions.
- Workload distribution.
- A table for the time taken for each task. *It will not affect your grade, so please be honest.*

Keep the document as simple as possible and do not include unnecessary information; we do not evaluate by word count!

The submission deadline is **WEEK 11. Exactly one day before the tutorial.**